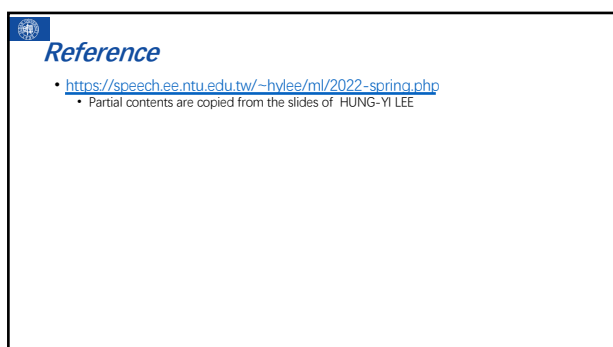




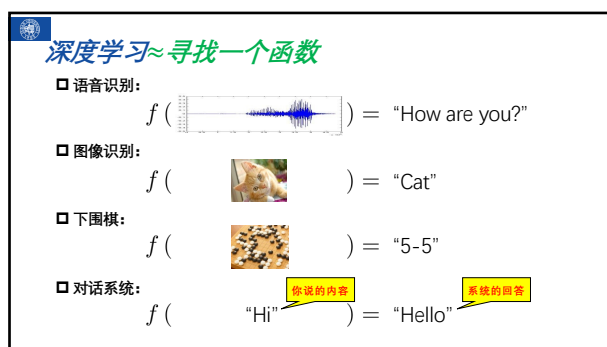
1



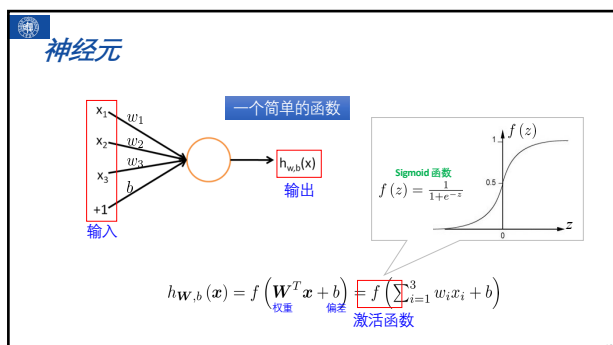
2



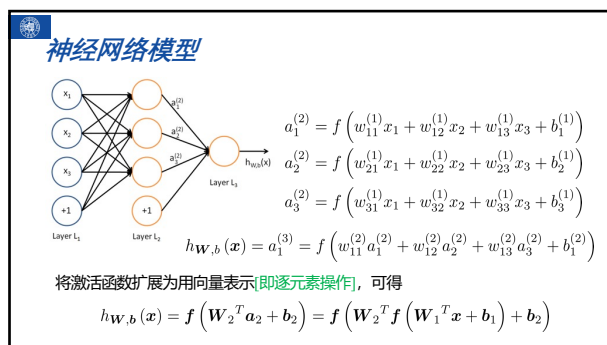
3



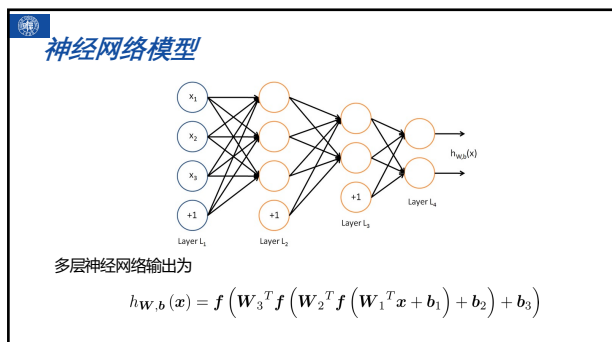
4



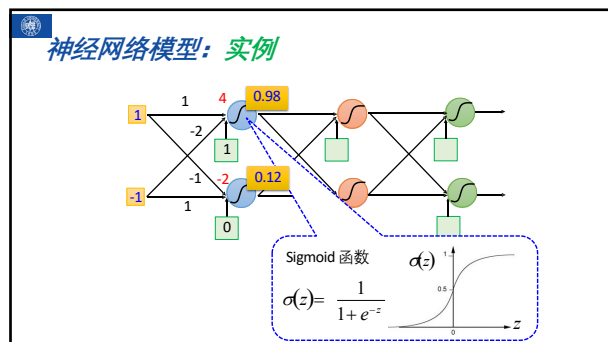
10



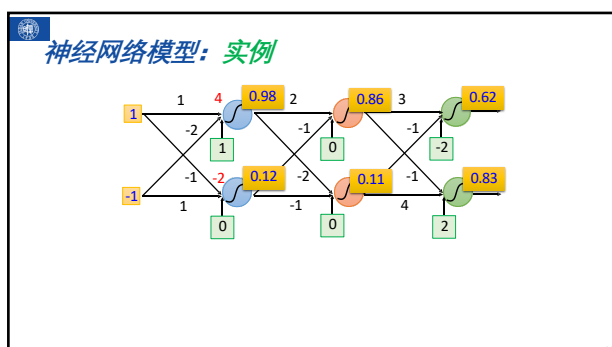
11



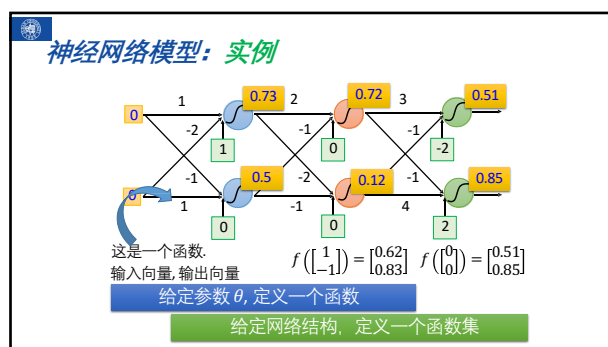
12



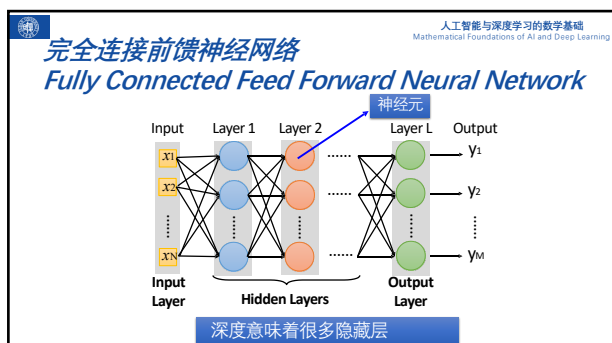
13



14



15



16



38

近年来涌现的人工智能/深度学习平台

- PyTorch
 - Facebook, 2017年1月
- TensorFlow
 - Google, 2015年11月
- MXNet: Amazon, 2015年9月
- CNTK: Microsoft, 2016年1月
- Caffe: 学术界, 2013年9月
- PaddlePaddle: 百度, 2016年8月

39

PyTorch主要特点

- 动态计算图
 - 动态计算图: 与静态计算图 (如 TensorFlow 早期版本) 不同, PyTorch 使用动态计算图, 这意味着计算图在运行时构建, 这使得调试更加灵活和直观。
 - 灵活性: 支持动态的控制流 (如条件语句、循环), 适合快速原型设计和研究。
- 自动求导 (Autograd)
 - 自动求导: PyTorch 内置了自动求导机制 (Autograd), 可以自动计算梯度, 非常适合深度学习中的反向传播。
 - 便捷性: 用户无需手动实现复杂的梯度计算, 只需定义前向传播即可。
- 强大的生态系统
 - 丰富的库和工具: 包括 torchvision (计算机视觉)、torchaudio (音频处理)、torchtext (自然语言处理) 等。
 - 社区支持: 拥有活跃的开发社区和丰富的教程、文档和支持资源。
- GPU 支持
 - 多平台支持: 支持 CPU 和 GPU 计算, 可以通过简单的 API 将张量和模型移动到 GPU 上加速训练。
 - 高效性: 利用 CUDA 技术最大化 GPU 的性能。
- Python 集成
 - 易于使用: 基于 Python, 语法简洁易懂, 适合科研人员 and 工程师。
 - 广泛的库支持: 与 NumPy 等科学计算库无缝集成。

40

PyTorch核心组件

- torch.Tensor
 - 张量 (Tensor): 类似于 NumPy 的 ndarray, 并且支持自动求导和 GPU 加速。
 - 操作: 提供各种数学运算、矩阵操作等功能。
- nn.Module
 - 模块 (Module): 用于定义神经网络层和整个模型。
 - 继承: 通过继承 nn.Module 类来创建自定义层和模型。
- autograd
 - 自动求导: 用于计算梯度, 支持链式法则。
- optim
 - 优化器 (Optimizer): 提供多种优化算法, 如 SGD、Adam、RMSprop 等。
 - 简单易用: 通过定义优化器对象并调用 step() 方法来更新参数。
- DataLoader
 - 数据加载器: 用于加载和预处理数据集, 支持批量处理、打乱数据等。
 - 方便集成: 与 torchvision 等库配合使用, 简化数据准备过程。

41

张量 (Tensor)

- 张量表示一个由数值组成的数组, 这个数组可能有多个维度。
 - 具有一个轴的张量对应数学上的向量 (vector);
 - 具有两个轴的张量对应数学上的矩阵 (matrix)。

向量创建

```
import torch
x = torch.arange(4)
y = torch.arange(6, dtype = torch.float32).reshape(2,3)
print(x)
print(y)

tensor([0, 1, 2, 3])
tensor([[0., 1., 2.],
        [3., 4., 5.]])
```

42

张量创建

```
import torch
import numpy as np
print(torch.empty(3,dtype=torch.int32))#int32
print(torch.zeros(2,3)) #float32
print(torch.randn(2,3))#float32
print(torch.tensor([1,2,0,3]))#float32
print(torch.tensor(np.array([3.,4.])))#float64

tensor([0, 0, 0], dtype=torch.int32)
tensor([[0., 0., 0.]])
tensor([[1.5152, 0.3698, 0.2815],
        [0.5603, 0.6311, 1.3469]])
tensor([1, 2, 3])
tensor([3., 4.], dtype=torch.float64)
```

43

张量属性

Tensor x: tensor([[1, 2, 3], [4, 5, 6]])

属性描述	例子	输出
形状 (Shape)	x.shape	Shape of Tensor x: torch.Size([2, 3])
数据类型 (dtype)	x.dtype	Dtype of Tensor x: torch.int64
设备 (Device)	x.device	Device of Tensor x: cpu 如果张量在 GPU 上, 则输出可能是 cuda:0 或类似的设备标识符。
维度 (Dimensions)	x.dim()	Dimensions of Tensor x: 2
元素数量 (Number of Elements)	x.numel()	Number of Elements in Tensor x: 6
是否在 CPU 上	x.is_cpu	Is Tensor on CPU: True 如果张量不在 CPU 上, 则输出为 False。
是否在 CUDA 上	x.is_cuda	Is Tensor on CUDA: False 如果张量在 CUDA 上, 则输出为 True。

44

运算符操作-按元素运算

```
x = torch.tensor([1., 2, 3, 4])
y = torch.tensor([2, 2, 2, 2])
x+y, x-y, x*y, x/y, x**y
```

```
(tensor([3., 4., 5., 6.]),
 tensor([-1., 0., 1., 2.]),
 tensor([2., 4., 6., 8.]),
 tensor([0.5000, 1.0000, 1.5000, 2.0000]),
 tensor([1., 4., 9., 16.]))
```

45

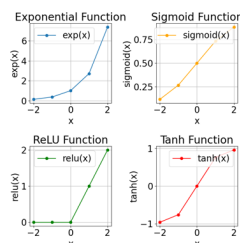
运算符操作-按元素运算

```
x = torch.tensor([-2., -1, 0, 1, 2])
x.exp(), x.sigmoid(), x.relu(), x.tanh()
```

```
(tensor([0.1353, 0.3679, 1.0000, 2.7183, 7.3891]),
 tensor([0.1192, 0.2689, 0.5000, 0.7311, 0.8808]),
 tensor([0., 0., 0., 1., 2.]),
 tensor([-0.9640, -0.7616, 0.0000, 0.7616, 0.9640]))
```

torch张量和numpy数组共享它们的底层内存

```
# 绘制 exp 图形
plt.subplot(2, 2, 1)
plt.plot(x.numpy(), exp_x.numpy(),
         label='exp(x)', marker='o')
plt.title('Exponential Function')
plt.xlabel('x')
plt.ylabel('exp(x)')
plt.grid(True)
plt.legend()
```



46

张量形状变换-squeeze和unsqueeze

```
# squeeze
x=torch.randn(1, 3, 4, 1)
x_squeeze = x.squeeze()
x.shape, x_squeeze.shape
```

```
(torch.Size([1, 3, 4, 1]),
 torch.Size([3, 4]))
```

```
# unsqueeze
x=torch.randn(3, 4)
x_0 = x.unsqueeze(0)
x_1 = x.unsqueeze(1)
x.shape, x_0.shape, x_1.shape
```

```
(torch.Size([3, 4]),
 torch.Size([1, 3, 4]),
 torch.Size([3, 1, 4]))
```

47

张量形状变换

```
# 形状变换
x_view = x.view(1, 3)
x_reshape = x.reshape(-1, 1)
x_transpose = z.transpose(0, 1)
```

```
x = torch.tensor([1, 2, 3])
z = torch.rand(2, 2)
```

```
(tensor([[[1, 2, 3]]],
        [2],
        [3]]),
 tensor([[[0.2802, 0.8455],
          [0.9093, 0.3397]])])
```

```
(tensor([1, 2, 3]),
 tensor([0.2802, 0.9093],
        [0.8455, 0.3397]))
```

48

掩码

```
# 掩码
masked_w1 = w[w>1.6]
masked_w2 = torch.where(w > 1.6, w, torch.tensor(0.0))
x_slice, masked_w1, masked_w2
```

```
w = torch.tensor([[1.0, 2.0],
                  [1.2, 1.6]])
tensor([2.]),
tensor([[0., 2.],
        [0., 0.]])
```

49

张量填充

```
# 填充
x_pad = torch.tensor([[1, 2], [3, 4]])
x_padded = F.pad(x_pad, pad=(1, 1, 1, 1), mode='constant', value=0)
x_pad, x_padded
```

```
(tensor([1, 2],
        [3, 4])),
 tensor([[0, 0, 0, 0],
        [0, 1, 2, 0],
        [0, 3, 4, 0],
        [0, 0, 0, 0]])
```

50

张量步长视图

```
x = torch.tensor([[1, 2, 3, 4], [5, 6, 7, 8]])
# 步长视图
x_as_strided = x.as_strided(size=(2, 2), stride=(2, 2))
# 切片
x_slice = x[:, :2]
x, x_as_strided, x_slice
```

```
(tensor([[1, 2, 3, 4],
          [5, 6, 7, 8]]),
 tensor([[1, 3],
          [3, 5]]),
 tensor([[1, 2],
          [5, 6]]))
```

51

运算符操作-张量连结 (concatenate)

-1 表示自动推断该维度的大小, 结果为 (2, 2)

```
X = torch.tensor([1, 2, 3, 4]).reshape(2, -1)
Y = torch.tensor([2, 2, 2, 2]).reshape(-1, 2)
torch.cat((X, Y), dim=0), torch.cat((X, Y), dim=1)
```

沿着第 0 维 (即行方向) 拼接张量 X 和 Y。

```
(tensor([[1, 2],
          [3, 4],
          [2, 2],
          [2, 2]]),
 tensor([[1, 2, 2, 2],
          [3, 4, 2, 2]]))
```

52

运算符操作-高维张量堆叠

```
# 创建一个形状为 (2, 2, 4) 的三维张量
tensor_3d_0 = torch.zeros(24).reshape(2, 2, 4)
tensor_3d_1 = torch.ones(16).reshape(2, 2, 4)

# 使用 torch.cat 将两个三维张量拼接成四维张量
tensor_4d_cat = torch.cat((tensor_3d_0.unsqueeze(0),
                           tensor_3d_1.unsqueeze(0)), dim=0)

tensor_3d_0, tensor_3d_1, tensor_4d_cat
```

*首先使用 `unsqueeze(0)` 在每个三维张量前面增加一个维度, 使其形状变为 (1, 2, 4, 5)。

*然后使用 `torch.cat` 沿着 `dim=0` 将这两个四维张量拼接起来, 形成一个新的四维张量, 形状为 (2, 2, 4, 5)。

```
(tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[1., 1., 1., 1.],
           [1., 1., 1., 1.]]],
        [[1., 1., 1., 1.],
           [1., 1., 1., 1.]]]),
```

```
[[[1., 1., 1., 1.],
   [1., 1., 1., 1.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

```
[[[0., 0., 0., 0.],
   [0., 0., 0., 0.]]],
 tensor([[[[0., 0., 0., 0.],
           [0., 0., 0., 0.]]],
        [[0., 0., 0., 0.],
           [0., 0., 0., 0.]]]),
```

53

向量矩阵运算

```
a = torch.tensor([4, 4])
b = torch.tensor([1, 1])
w = torch.tensor([[1, 1], [2, 2], [3, 3]])
m = torch.tensor([[1, 2], [3, 4]])
```

```
(tensor(8),
 tensor([2, 4, 6]),
 tensor([[ 4, 6],
          [ 8, 12],
          [12, 18]]))
```

```
torch.dot(a, b) # 向量点积
torch.mv(w, b) # 矩阵向量乘法
torch.mm(w, m) # 矩阵乘法
torch.matmul(w, m) # 通用张量乘法
```

54

计算梯度-backward

```
import torch

# 创建一个需要计算梯度的张量
x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)

# 定义一个简单的计算图
y = (x * 2).sum()

# 计算梯度
y.backward()
```

```
# 获取梯度
print("张量 x 的梯度:", x.grad)
```

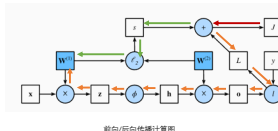
张量 x 的梯度: tensor([2., 2., 2.])

```
若: y = (x ** 2 * 2).sum()
张量 x 的梯度: tensor([ 4., 8., 12.])
若: y = (x ** 2).mean()
张量 x 的梯度: tensor([0.6667, 1.3333, 2.0000])
```

55

基于计算图的前向后向计算

前向传播 (forward propagation或forward pass) 指的是: 按顺序 (从输入层到输出层) 计算和存储神经网络中每层的结果。**后向传播** (backward propagation或backward pass) 指的是: 按逆序 (从输出层到输入层) 计算和存储神经网络中每层的梯度。



前向/后向传播计算图

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x},$$

$$\mathbf{h} = \phi(\mathbf{z}).$$

$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h},$$

$$L = l(\mathbf{o}, \mathbf{y}).$$

$$s = \frac{\lambda}{2} (\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2),$$

$$J = L + s.$$

输入样本为 $\mathbf{X}$, 隐藏层的权重 $\mathbf{W}$, 中间变量 $\mathbf{Z}$, 激活函数 ϕ , 样本标签为 $\mathbf{y}$, 损失函数为 $\mathbf{L}$, $\mathbf{s}$ 为正则化项, $\mathbf{J}$ 为目标函数

56

梯度下降算法- Gradient Descent

- 模型
 - 通过继承 nn.Module 类来创建自定义层和模型。
- 数据
 - 使用 DataLoader 加载和预处理数据集，支持批量处理、打乱数据等。
- 优化器
 - 使用 torch.optim 提供多种优化算法，如 SGD、Adam、RMSprop 等，并调用 step() 方法来更新参数。
 - 调用 tensor.backward() 进行自动求导：用于计算梯度，支持链式法则。

```
# 训练模型
for epoch in range(num_epochs):
    # 前向传播
    y_pred = model(X)

    # 计算损失
    loss = criterion(y_pred, y)

    # 反向传播
    optimizer.zero_grad()
    loss.backward()

    # 更新权重和偏置
    optimizer.step()
```

58

梯度下降算法- 线性回归

假设我们有一个线性模型：
 $y_{pred} = WX + b$

其中：

- X 是输入数据矩阵，形状为 (m, n) ，其中 m 是样本数量， n 是特征数量。
- W 是权重矩阵，形状为 $(n, 1)$ 。
- b 是偏置向量，形状为 $(1, 1)$ 。
- y_{pred} 是预测值矩阵，形状为 $(m, 1)$ 。

损失函数

均方误差 (MSE) 损失函数定义为：

$$L = \frac{1}{2} \sum_{i=1}^m (y_{pred,i} - y_i)^2$$

其中：

- $y_{pred,i}$ 是第 i 个样本的预测值。
- y_i 是第 i 个样本的真实值。
- m 是样本总数。

$$\frac{\partial L}{\partial W} = \frac{1}{2} \sum_{i=1}^m (y_{pred,i} - y_i) \cdot X^T$$

$$\frac{\partial L}{\partial b} = \frac{1}{2} \sum_{i=1}^m (y_{pred,i} - y_i)$$

- 模型
 - $y_{pred} = np.dot(X, W) + b$
- 损失
 - $loss = np.mean((y_{pred} - y) ** 2)$
- 优化
 - 反向传播
 - $dw = 2 * np.mean((y_{pred} - y) * X, axis=0, keepdims=True) * T$
 - $db = 2 * np.mean((y_{pred} - y), axis=0, keepdims=True)$
 - 更新权重和偏置
 - $W -= learning_rate * dw$
 - $b -= learning_rate * db$

- 数据: $[[x1, x2, y]]$
 - 线性可分: $[[0,0,0],[0,1,1],[1,0,0],[1,1,1]]$
 - XOR: $[[0,0,0],[0,1,1],[1,0,1],[1,1,0]]$ // 不可分

59

梯度下降算法-线性回归

```
# 训练模型
for epoch in range(num_epochs):
    # 前向传播
    y_pred = np.dot(X, W) + b

    # 计算损失
    loss = np.mean((y_pred - y) ** 2)

    # 反向传播
    dw = np.mean((y_pred - y) * X, axis=0, keepdims=True) * T
    db = np.mean((y_pred - y), axis=0, keepdims=True)

    # 更新权重和偏置
    W -= learning_rate * dw
    b -= learning_rate * db
```

numpy实现: 写详细代码

```
# 训练模型
for epoch in range(num_epochs):
    # 前向传播
    y_pred = model(X)

    # 计算损失
    loss = criterion(y_pred, y)

    # 反向传播
    optimizer.zero_grad()
    loss.backward()

    # 更新权重和偏置
    optimizer.step()
```

torch实现: 写model类

60

梯度下降算法-线性回归

```
# 定义模型类
class Perceptron(nn.Module):
    def __init__(self, input_size, output_size=1):
        super(Perceptron, self).__init__()
        self.linear = nn.Linear(input_size, output_size)

    def forward(self, x):
        return self.linear(x)

# 初始化模型、损失函数和优化器
model = Perceptron(input_size=2, output_size=1)
# 损失函数的输入是模型输出和参考答案
criterion = nn.MSELoss()
# 优化器和模型参数绑定
optimizer = optim.SGD(model.parameters(), learning_rate)
```

torch实现: 写model类

61

回归问题

```
# 定义一个新模型
class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size=1):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size) # 输入到隐藏层
        self.relu = nn.ReLU() # ReLU激活函数
        self.fc2 = nn.Linear(hidden_size, output_size) # 隐藏层到输出

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        return out

# 初始化模型
model = SimpleNN(input_size=2, hidden_size=4, output_size=1)
```

torch实现: 换个模型

62

分类问题

```
# 定义一个新模型
class MLP(nn.Module):
    def __init__(self, input_size, hidden_size, output_size=1):
        super(MLP, self).__init__()
        self.fc1 = nn.Linear(input_size, hidden_size) # 输入到隐藏层
        self.relu = nn.ReLU() # ReLU激活函数
        self.fc2 = nn.Linear(hidden_size, output_size) # 隐藏层到输出
        self.sigmoid = nn.Sigmoid() # sigmoid激活函数

    def forward(self, x):
        out = self.fc1(x)
        out = self.relu(out)
        out = self.fc2(out)
        out = self.sigmoid(out)
        return out

# 初始化模型
model = MLP(input_size=2, hidden_size=4, output_size=1)
criterion = nn.BCELoss() # 使用二元交叉熵损失函数
```

63

随机梯度下降SGC和DataLoader

随机梯度下降 (Stochastic Gradient Descent, SGD) 是一种优化算法，用于最小化损失函数。它通过使用训练数据的一个样本或一个小批次来近似整个数据集的梯度，并根据这个近似的梯度更新模型参数。这种方法可以加速收敛过程，并有助于跳出局部最优解。

```
# 训练模型
for epoch in range(num_epochs):
    for inputs, targets in dataloader:
        # 前向传播
        y_pred = model(inputs)
```

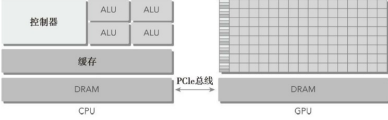
DataLoader 是一个用于加载数据的实用工具。它提供了对数据集的迭代器接口，并支持**自动批量化**、**打乱数据**和**多线程数据加载**等功能。

```
# 创建TensorDataset和DataLoader
dataset = TensorDataset(X, y)
dataloader = DataLoader(dataset, batch_size = 4, shuffle = True)
```

64

异构计算

- 一个典型的异构计算节点包括两个或多个核 CPU 插槽和两个或更多个众核 GPU
- GPU不是一个独立运行的平台而是**CPU的协处理器**。因此，GPU必须通过PCIe总线与基于CPU的主机相连来进行操作，如图所示。这就是为什么CPU所在的位置被称作**主机端**而GPU所在的位置被称作**设备端**。



65

GPU训练推理

```
# 1. 检查GPU是否可用 ( cuda, mps或...)
use_cuda = torch.cuda.is_available()
if use_cuda:
    device = torch.device("cuda")
else:
    device = torch.device("cpu")

# 2. 把数据和模型参数传输到GPU上
X = X_cpu.to(device)
y = Y_cpu.to(device)
model.to(device)

# 3. 模型推理: 把结果传输到CPU上
with torch.no_grad(): #块内的所有操作都不会计算梯度。
    predicted = model(X)
    predicted = (predicted > 0.5).int()
    print('Predicted:', predicted.cpu().numpy().flatten())
    print('True:', y.cpu().numpy().flatten())
```

66

其他

```
# 设置随机种子以保证结果可重复
torch.manual_seed(43)
np.random.seed(43)
```

创建张量	形状变换	切片	掩码	拼接
堆叠	分割	转置	展开	添加维度
连续存储	步长视图	填充	类型转换	设备移动
梯度计算	求和	均值	最大值/最小值	Top-K

在PyTorch中进行多GPU训练可以显著加速模型训练，尤其是在处理大规模数据集和复杂模型时。PyTorch提供了多种多GPU训练的方式，包括**数据并行 (Data Parallelism)** 和 **分布式数据并行 (Distributed Data Parallelism, DDP)**。

67